VPR Tutorial Architecture Guide

A Practical Codebase Walkthrough for New Developers

Project Documentation

April 22, 2026

Purpose

This document is a practical onboarding guide to the current codebase. It is written for a junior developer who wants to understand:

- where the code starts,
- how the original benchmark pipeline and the newer live pipeline relate,
- which files are responsible for which parts of the system,
- and where to make changes when extending the project.

This guide is intentionally tutorial-style: it explains the important flow first, then tours the key files with short code snippets and commentary.

1 Big Picture

The repository now has two production-relevant paths that share the same VPR idea.

- **Benchmark path:** load a dataset, extract descriptors, compute a similarity matrix, apply matching, then compute metrics and visualizations.
- **Live path:** build a reusable reference map, then localize camera frames or video frames against that map in real time.

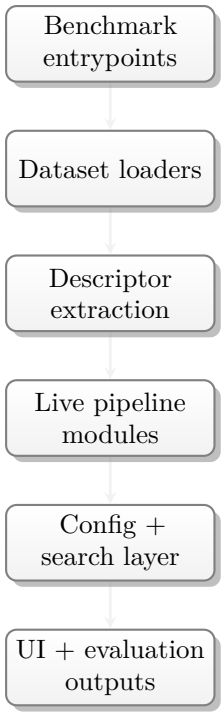

Figure 1: Repository overview: the benchmark and live paths share descriptor ideas, while the live path adds configuration, persisted maps, and pluggable search.

2 Where To Start Reading

If you are new to the project, read these files in this order:

1. `demo.py`
2. `datasets/load_dataset.py`
3. `matching/matching.py`
4. `evaluation/metrics.py`
5. `test_campus_dataset.py`
6. `live_vpr_test.py`
7. the modules under `live_vpr/`

This order is useful because it teaches the simple benchmark flow first and then the more modular live system.

3 Architecture Diagrams

3.1 Benchmark Flow

Figure 2: The benchmark path computes a full similarity matrix and then evaluates it against ground truth.

3.2 Live Flow

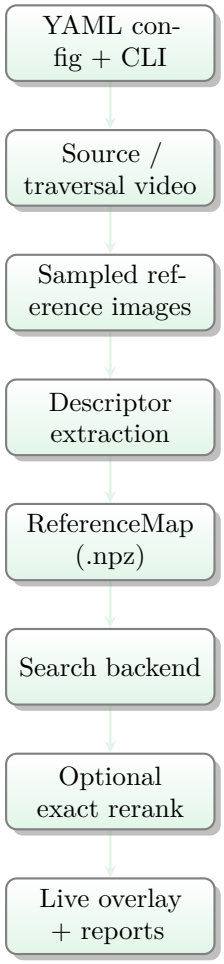

Figure 3: The live path builds a reusable map once and then performs repeated single-frame localization against it.

3.3 Module-Level Ownership

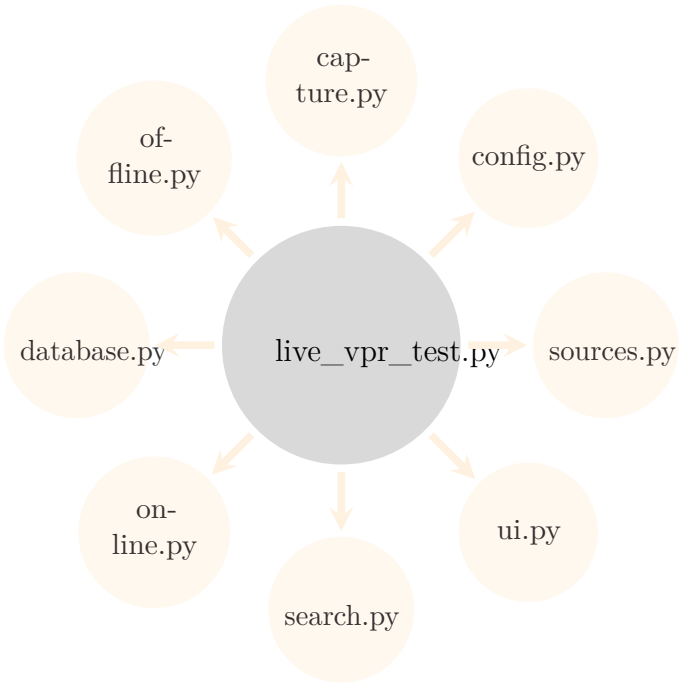

Figure 4: Module-level view of the live system. `live_vpr_test.py` coordinates the workflow; the logic lives in smaller modules.

3.4 Artifacts and Persistence Flow

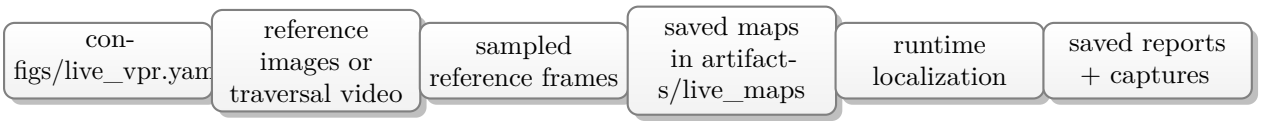

Figure 5: Important filesystem artifacts. The live system relies on persisted map files rather than repeatedly scanning raw folders during runtime.

4 Code Tour

4.1 Execution Modes in `live_vpr_test.py`

The live CLI has grown into a mode-based orchestrator. This is the high-level contract for each mode:

Mode	Primary modules	Output / side effect
`build_map`	`live_vpr/offline.py`, `live_vpr/database.py`	Builds a map from image directory and saves `*.npz`.
`build_live_map`	`live_vpr/capture.py`, `live_vpr/offline.py`	Records traversal video (or reuses video), samples frames, then writes map.
`live`	`live_vpr/sources.py`, `live_vpr/online.py`, `live_vpr/ui.py`	Runs camera/stream localization with interactive controls.
`video`	`live_vpr/online.py`, `live_vpr/ui.py`	Runs localization on prerecorded video stream.
`check_source`	`live_vpr/sources.py`	Probes one source and prints backend/frame metadata.
`list_sources`	`live_vpr/sources.py`	Scans camera indices, optionally saves preview frames.
`alias_modes` (save/list/delete)	`live_vpr/sources.py`	Persists aliases in `artifacts/live_vpr_sources.json`.

4.2 `demo.py`

What it does. This is the original tutorial entrypoint. It demonstrates the full benchmark evaluation loop on the built-in datasets.

Why it matters. If you understand this file, you understand the basic VPR idea used everywhere else in the repo.

Listing 1: Core benchmark pattern in `demo.py`

```
imgs_db, imgs_q, GThard, GTsoft = dataset.load()

db_D_holistic = feature_extractor.compute_features(imgs_db)
q_D_holistic = feature_extractor.compute_features(imgs_q)

db_D_holistic = db_D_holistic / np.linalg.norm(db_D_holistic, axis=1, keepdims=True)
q_D_holistic = q_D_holistic / np.linalg.norm(q_D_holistic, axis=1, keepdims=True)
S = np.matmul(db_D_holistic, q_D_holistic.transpose())
```

How it works. The file loads images, converts them into descriptors, normalizes those descriptors, and computes the similarity matrix S. Everything later in the benchmark path depends on S.

4.3 `datasets/load_dataset.py`

What it does. This file owns dataset loading and ground-truth generation for:

- GardensPoint
- StLucia
- SFU
- the custom `CampusDataset` (day/night campus data)

Most useful idea. All loaders return the same interface: reference images, query images, strict ground truth, and soft ground truth.

Listing 2: Shared dataset contract

```
class Dataset(ABC):
    @abstractmethod
    def load(self) -> Tuple[List[np.ndarray], List[np.ndarray], np.ndarray, np.ndarray]:
        pass
```

How it works. This shared contract is what lets `demo.py` and `test_campus_dataset.py` swap datasets without changing the rest of the evaluation code.

Important current behavior for the campus set.

- Database images come from `custom_dataset/day_images`.
- Query images come from `custom_dataset/night_images`.
- Any query with `-npm`, `npm*`, or `PXL_*` is treated as no-perfect-match and does not receive a hard GT positive.
- Soft GT is created by vertical dilation of hard GT with a 5x1 kernel (tolerance in nearby day indices).

Listing 3: Campus ground-truth logic

```
if '-npm' in night_name.lower() or night_name.startswith('npm'):
    continue
if night_name.startswith('PXL_'):
    continue

match = re.match(r'(image\d+)', night_name)
if match:
    base_name = match.group(1)
    if base_name in day_names:
        i = day_names.index(base_name)
        GThard[i, j] = True
```

How it works. The campus loader uses filenames to decide which night images truly match day images and which should be treated as no-match queries.

4.4 `matching/matching.py`

What it does. This file converts similarity scores into binary match decisions.

Listing 4: Two core matching strategies

```
def best_match_per_query(S: np.ndarray) -> np.ndarray:
    i = np.argmax(S, axis=0)
    j = np.int64(range(len(i)))
    M = np.zeros_like(S, dtype='bool')
    M[i, j] = True
    return M

def thresholding(S: np.ndarray, thresh: Union[str, float]) -> np.ndarray:
    if thresh == 'auto':
        mu = np.median(S)
        sig = np.median(np.abs(S - mu)) / 0.675
        thresh = norm.ppf(1 - 1e-6, loc=mu, scale=sig)
    return S >= thresh
```

How it works. One strategy always chooses exactly one match per query. The other uses a threshold so a query can have zero, one, or many matches depending on confidence.

4.5 `evaluation/metrics.py`

What it does. This file computes the benchmark metrics.

Listing 5: How PR curves are built

```
for i in np.linspace(startV, endV, n_thresh):
    B = S >= i
    TP = np.count_nonzero(GT & B)
    FP = np.count_nonzero((~GT) & B)
    P.append(TP / (TP + FP))
    R.append(TP / GTP)
```

How it works. Instead of choosing one threshold once, the code sweeps across many thresholds and records precision and recall at each one. That produces the PR curve and the AUC.

4.6 `test_campus_dataset.py`

What it does. This file adapts the benchmark pipeline to the custom campus dataset.

Why it matters. It shows how to add a custom dataset without rewriting the whole evaluation stack.

Listing 6: Campus script follows the same flow as `demo.py`

```
dataset = CampusDataset(destination=args.dataset_dir)
imgs_db, imgs_q, GThard, GTsoft = dataset.load()

db_D_holistic = feature_extractor.compute_features(imgs_db)
q_D_holistic = feature_extractor.compute_features(imgs_q)
S = np.matmul(db_D_holistic, q_D_holistic.transpose())
```

How it works. The structure is intentionally similar to `demo.py`. The biggest difference is the campus loader and the saved campus-specific output files.

4.7 `live_vpr_test.py`

What it does. This is the top-level CLI for the live pipeline.

Why it matters. It does not implement most logic itself; it orchestrates the modules under `live_vpr/`.

Listing 7: The live CLI wires together modular components

```
from live_vpr import (
    LiveDisplay,
    LiveLocalizer,
    LiveReferenceRecorder,
    MapBuildConfig,
    MapBuilder,
    OpenCVFrameSource,
    SearchConfig,
    load_reference_map,
    parse_args_with_config,
)
```

How it works. This file is best understood as the glue layer. If you want to add a new mode or a new CLI flag, this is where you start.

Current branch detail. It now also includes:

- mode aliases for backward compatibility (for example, `build_db` → `build_map`),
- YAML configuration loading before CLI overrides,
- source alias persistence commands,
- optional per-inference report image export,
- configurable inference cadence via `-process_fps`,
- pluggable search backends for runtime map lookup.

4.8 `live_vpr/config.py` and `configs/live_vpr.yaml`

What they do. These files define how the live pipeline loads defaults from YAML and then lets CLI flags override those defaults.

Listing 8: Config-first argument parsing

```
args = parse_args_with_config(parser)
```

How it works. The config loader flattens the YAML tree, validates that every key corresponds to a real parser destination, applies those values as parser defaults, and then parses the final command line. This keeps runtime tuning modular instead of scattering hardcoded constants across the CLI.

4.9 `live_vpr/offline.py`

What it does. This file owns reference-map creation from images.

Listing 9: Core map-building loop

```
images = [_load_rgb_image(path, target_size) for path in image_paths]
descriptors = compute_global_descriptors(self.extractor, images)
descriptors = normalize_descriptors(descriptors)
reference_map = ReferenceMap(
    descriptors=descriptors,
    image_paths=[str(path.resolve()) for path in image_paths],
    metadata=metadata,
)
```

How it works. A map is just a normalized descriptor matrix plus metadata and image paths. This is the main transition from "folder of images" to "runtime-ready live map".

4.10 `live_vpr/database.py`

What it does. This file defines the saved map format and serialization.

Listing 10: ReferenceMap is the live system's central data object

```
@dataclass
class ReferenceMap:
    descriptors: np.ndarray
    image_paths: list[str]
    metadata: dict[str, Any]
```

How it works. The live runtime does not read raw folders directly. It loads this object from disk and then does similarity lookup against `descriptors`.

4.11 `live_vpr/capture.py`

What it does. This file owns traversal recording and frame sampling.

Listing 11: Recording first, building later

```
if recording:
    writer.write(frame)
    frame_count += 1

status_frame = self._render_overlay(frame, recording, frame_count)
```

Listing 12: Sampling a traversal video into reference frames

```
should_sample = current_time_s + (0.5 / source_fps) >= next_sample_time_s
if should_sample:
    if config.save_scale < 1.0:
        frame_to_save = cv2.resize(frame, (new_width, new_height), interpolation=cv2.
    INTER_AREA)
    cv2.imwrite(str(output_path), frame_to_save)
    saved_paths.append(str(output_path))
    next_sample_time_s += sample_interval_s
```

How it works. Recording and sampling are intentionally separate. That makes it easy to rebuild a map later with a different sampling rate, and the saved-frame downsampling option now lets the project keep the sampled reference set smaller on disk.

4.12 `live_vpr/online.py`

What it does. This file owns the runtime localization step.

Listing 13: The live localization core

```
descriptor = compute_global_descriptors(self.extractor, [rgb_image])
descriptor = descriptor / (np.linalg.norm(descriptor, axis=1, keepdims=True) + 1e-8)
search_result = self.search_backend.search(descriptor[0], top_k=self.top_k)
```

How it works. The live phase still performs VPR in the same conceptual way as the benchmark path, but the lookup is no longer hardcoded to a full linear scan. `live_vpr/online.py` now delegates retrieval to a search backend and only concerns itself with frame encoding, thresholding, and packaging the result.

4.13 `live_vpr/search.py`

What it does. This file owns scalable reference-map lookup.

Listing 14: Search backends are created from one configuration object

```
search_config = SearchConfig.from_namespace(args)
self.search_backend = create_search_backend(reference_map.descriptors, self.search_config
    )
```

Current supported backends.

- `exact`: full scan over the descriptor matrix with partial top-k selection,
- `hnsw`: approximate nearest-neighbor retrieval using HNSW,
- `faiss_ivf_flat`: FAISS IVF search with flat residual storage,
- `faiss_ivf_pq`: FAISS IVF search with product quantization for larger maps.

How they are currently implemented.

- `exact` uses the descriptor matrix directly and applies a partial top-k selection instead of fully sorting every score.
- `hnsw` builds an HNSW index once, then queries that graph for a shortlist of likely candidates.
- `faiss_ivf_flat` trains a FAISS coarse quantizer, probes only the most relevant clusters, and keeps exact vectors inside them.
- `faiss_ivf_pq` uses the same FAISS cluster idea but stores vectors in compressed PQ form to save memory.

Listing 15: Approximate search can still be followed by exact reranking

```
if self.config.rerank:
    indices, scores = _rerank_exact(self.descriptors, query, candidate_indices, top_k)
```

Why reranking exists. ANN methods are fast because they avoid scoring every map descriptor, but their returned order is only approximate. Reranking recomputes the exact scores on the candidate shortlist so the final top-k order and thresholding score are closer to the true cosine result.

A tiny intuition example.

- If the map has only a few hundred references, `exact` is often simplest and perfectly fine.
- If the map grows to tens of thousands of references, `hnsw` can quickly return a shortlist like $\{E, F, D\}$ instead of scanning the whole map.
- `faiss_ivf_flat` is strong when the map is large and static because it searches a few relevant clusters rather than the full matrix.
- `faiss_ivf_pq` helps when memory becomes part of the problem, not just query latency.

How it works. This layer is what lets the repo move from small-map exact matching to larger-map ANN search without changing the rest of the live localization flow. The localizer still only asks for "top-k matches"; the search layer decides how to retrieve them.

4.14 `live_vpr/sources.py`

What it does. This file abstracts webcams, stream URLs, and source aliases.

Listing 16: Alias resolution happens before capture opens

```
def parse_capture_source(source: str | int, aliases_path: str | None = None) -> str | int
    :
    aliases = load_source_aliases(aliases_path)
    while source in aliases and source not in visited:
        source = aliases[source].strip()
    return int(source) if source.isdigit() else source
```

How it works. This lets the rest of the pipeline treat `0`, `phone`, and `http://...` as different ways of naming the same idea: a capture source.

4.15 `live_vpr/ui.py`

What it does. This file owns the live overlay and the saved per-inference report images.

Listing 17: The UI has separate live and export render paths

```
display = LiveDisplay(reference_map=reference_map, show_top_k=not args.hide_top_k)
rendered = display.render(...)
export_canvas = display.render_inference_report(...)
```

How it works. The live overlay is optimized for real-time display, while the saved report image is optimized for readability and documentation.

4.16 `scripts/live_vpr_cli.sh`

What it does. This is a thin shell wrapper around `live_vpr_test.py`.

Listing 18: The launcher maps short commands to Python modes

```
record-map)
  exec "$PYTHON_BIN" "$ENTRYPOINT" \
    --mode build_live_map \
    --descriptor "$DEFAULT_DESCRIPTOR" \
    --map_path "$DEFAULT_MAP_PATH" \
    --source "$DEFAULT_SOURCE" \
```

```
    --recording_path "$DEFAULT_RECORDING_PATH" \
    --capture_dir "$DEFAULT_CAPTURE_DIR" \
    --sample_fps "$DEFAULT_SAMPLE_FPS" \
    "$@"
```

How it works. The shell script does not contain VPR logic. It just shortens the commands and provides convenient environment-variable defaults.

5 Worked Example A: Benchmark Evaluation (10-image dry run)

This section is intentionally contrived. The goal is to make the evaluation math intuitive, not to mimic real descriptor distributions.

5.1 Setup

Assume:

- 10 database images: $D_1, D_2, \ldots, D_{10}$ (day/reference),
- 10 query images: $Q_1, Q_2, \ldots, Q_{10}$ (night/query),
- queries Q_1 to Q_7 have true matches,
- queries Q_8 to Q_{10} are no-perfect-match cases (similar to `-npm` behavior).

5.2 Step 1: Build hard ground truth GT_{hard}

Here is the hard GT matrix (rows are database images, columns are queries):

Listing 19: Contrived hard GT matrix (1 = true match)

```
          Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8 Q9 Q10
D1         1  0  0  0  0  0  0  0  0   0
D2         0  1  0  0  0  0  0  0  0   0
D3         0  0  1  0  0  0  0  0  0   0
D4         0  0  0  1  0  0  0  0  0   0
D5         0  0  0  0  1  0  0  0  0   0
D6         0  0  0  0  0  1  0  0  0   0
D7         0  0  0  0  0  0  1  0  0   0
D8         0  0  0  0  0  0  0  0  0   0
D9         0  0  0  0  0  0  0  0  0   0
D10        0  0  0  0  0  0  0  0  0   0
```

Interpretation:

- Q_1 matches only D_1, Q_2 matches only D_2, ..., Q_7 matches only D_7.
- Q_8, Q_9, Q_{10} have no true match.

5.3 Step 2: Build soft ground truth GT_{soft}

In this repository, soft GT is a dilated version of hard GT (for example, with a vertical kernel). Intuition:

- if D_4 is the strict match for Q_4,
- soft GT may also mark nearby rows (D_2 to D_6) as acceptable overlap.

For this toy dry run, we focus metric interpretation on hard positives and use soft GT as an *ignore band* for near-miss evaluations (same concept as `evaluation/metrics.py`).

5.4 Step 3: Descriptor extraction and similarity matrix S

After descriptor extraction and normalization, suppose the key similarities are:

Query	Best score	Interpretation
Q_1	$S(D_1, Q_1) = 0.93$	Correct strong match
Q_2	$S(D_2, Q_2) = 0.90$	Correct strong match
Q_3	$S(D_3, Q_3) = 0.88$	Correct strong match
Q_4	$S(D_4, Q_4) = 0.86$	Correct strong match
Q_5	$S(D_5, Q_5) = 0.84$	Correct match, slightly weaker
Q_6	$S(D_6, Q_6) = 0.82$	Correct match, slightly weaker
Q_7	$S(D_7, Q_7) = 0.80$	Correct match, weaker but still right
Q_8	$S(D_3, Q_8) = 0.81$	False high-confidence match (no GT)
Q_9	$S(D_6, Q_9) = 0.79$	False high-confidence match (no GT)
Q_{10}	$S(D_2, Q_{10}) = 0.77$	False high-confidence match (no GT)

5.5 Step 4: Matching outputs

Single-best-match (M_1): choose argmax per query.

- Predictions: 10 total (one per query).
- True positives: 7 (for $Q_1..Q_7$).
- False positives: 3 (for $Q_8..Q_{10}$, which should be unmatched).
- Precision $= 7/10 = 0.70$.

Thresholding (M_2): mark all entries with $S \geq t$.

5.6 Step 5: Dry-run precision/recall at multiple thresholds

Let the number of ground-truth positives be $GTP = 7$.

Threshold t	TP	FP	Precision	Recall
0.85	4	0	$4/(4+0) = 1.00$	$4/7 = 0.571$
0.80	7	1	$7/(7+1) = 0.875$	$7/7 = 1.00$
0.75	7	3	$7/(7+3) = 0.70$	$7/7 = 1.00$

This is exactly what `createPR()` does conceptually: sweep thresholds from high to low, collect (R, P) pairs, then plot the PR curve.

How TP and FP are counted in each row (important). For each threshold t, the algorithm first creates a binary prediction matrix

$$B_t = (S \geq t)$$

with the same shape as the ground-truth matrix. Then:

$$TP = |GT \wedge B_t|, \qquad FP = |\neg GT \wedge B_t|$$

where $|\cdot|$ means "number of true entries."

In this toy setup, you can read each row as follows:

- $t = 0.85$**:** only 4 true-match scores are above threshold, and no no-match query crosses threshold. So $TP = 4$, $FP = 0$.
- $t = 0.80$**:** all 7 true matches now pass threshold, but one no-match query also passes (a spurious high score). So $TP = 7$, $FP = 1$.
- $t = 0.75$**:** true matches remain 7, and now three no-match queries pass threshold. So $TP = 7$, $FP = 3$.

Lowering the threshold usually increases recall first, then starts to add false positives.

Link to the actual code in this repository. The benchmark implementation in `evaluation/metrics.py` performs this exact sequence at each threshold:

Listing 20: Per-threshold counting logic used in createPR()

```
B = S >= i
TP = np.count_nonzero(GT & B)
FP = np.count_nonzero((~GT) & B)
P.append(TP / (TP + FP))
R.append(TP / GTP)
```

5.7 Step 6: AUC, $R@100P$, and $R@K$

- **AUC:** area under the PR curve. Larger is better because high precision is maintained across more recall levels.
- $R@100P$**:** largest recall where precision is exactly 1.0. In this toy table, $R@100P \approx 0.571$ from $t = 0.85$.
- $R@K$**:** for each query with at least one true match, check whether the true database index appears in top-K similarities.

Small intuitive descriptions for this project.

- **Precision:** "When the system says match, how often is it correct?" High precision matters when false alarms are expensive (for example, wrong place recognition).
- **Recall:** "Of all real matches that exist, how many did we recover?" High recall matters when missing true places is expensive.

- **AUC (PR-AUC):** single summary of the precision-recall tradeoff over many thresholds. Better descriptor/matching pipelines keep precision high while recall grows, increasing AUC.
- $R@100P$**:** best recall you can get while making *no* false positives. Good for safety-critical operating points.
- $R@K$**:** ranking quality. If the correct place is in top-K, we count it as success. Useful for retrieval interfaces and human-in-the-loop systems.

For this toy setup (assuming all matched queries contain their true pair within top-5):

- $R@1 = 7/7 = 1.0$,
- $R@5 = 7/7 = 1.0$,
- $R@10 = 7/7 = 1.0$.

The important nuance is that $R@K$ ignores no-match queries by design (same as `recallAtK()`), while PR-based metrics explicitly penalize false positives on no-match queries.

5.8 Intuition summary

- GT_{hard} defines strict correctness.
- GT_{soft} defines tolerance/ignore regions for near overlaps.
- S converts descriptors into comparable scores.
- Matching strategy turns scores into decisions.
- Evaluation checks how good those decisions are across operating points.

6 Worked Example B: Live flow and database flow (contrived)

This dry run explains how map creation and frame-by-frame localization interact in practice.

6.1 Step 1: Build a tiny reference map

Assume 5 sampled reference frames:

- `ref_0000.jpg` (library entrance)
- `ref_0001.jpg` (corridor turn)
- `ref_0002.jpg` (cafeteria front)
- `ref_0003.jpg` (engineering gate)
- `ref_0004.jpg` (parking corner)

Offline pipeline (`live_vpr/offline.py` + `live_vpr/database.py`):

1. Load and resize all 5 images to target size (for example, 640×480).
2. Extract global descriptors with chosen model (for example, CosPlace).

3. Normalize descriptor rows.

4. Store in `ReferenceMap{descriptors, image_paths, metadata}`.

5. Serialize to `artifacts/live_maps/campus_tiny.npz`.

Map artifact contents are conceptually:

Listing 21: Contrived ReferenceMap payload

```
descriptors: shape (5, 512), float32, L2-normalized
image_paths: [
  ".../ref_0000.jpg", ".../ref_0001.jpg", ..., ".../ref_0004.jpg"
]
metadata: {
  "descriptor": "CosPlace",
  "target_size": [640, 480],
  "num_images": 5,
  "descriptor_dim": 512,
  "created_utc": "..."
}
```

6.2 Step 2: Online localization for incoming frames

Assume threshold is 0.75 and top-$k = 3$.

Frame A (actual location near library entrance):

- Similarities to map rows: $[0.91, 0.63, 0.44, 0.20, 0.11]$.
- Best index: 0 (`ref_0000.jpg`), best score: 0.91.
- Since $0.91 \geq 0.75$, decision is **recognized**.

Frame B (challenging viewpoint / motion blur):

- Similarities: $[0.58, 0.62, 0.59, 0.57, 0.55]$.
- Best index: 1, best score: 0.62.
- Since $0.62 < 0.75$, decision is **unknown**.

Frame C (near engineering gate):

- Similarities: $[0.22, 0.30, 0.41, 0.83, 0.49]$.
- Best index: 3, best score: 0.83.
- Since $0.83 \geq 0.75$, decision is **recognized**.

6.3 Step 2.5: What the live UI "Score" means

In live mode, the displayed score is the **maximum cosine similarity** between the current query-frame descriptor and all map descriptors:

$$\text{score} = \max_j \left(d_q^\top d_j\right)$$

where d_q is the normalized descriptor of the current frame and d_j are normalized descriptors stored in the map.

Interpretation in this project:

- Score close to 1.0: very strong visual agreement with some reference place.
- Mid score (for example, 0.55 to 0.75): uncertain/ambiguous match.
- Low score: likely unknown place or severe appearance change.

Decision rule used online:

- if `score >= threshold`: show **MATCH** (recognized),
- otherwise: show **UNKNOWN**.

So the live score is a *runtime confidence signal*, not a full offline metric. Offline metrics (Precision/Recall/AUC/R@K) require dataset-level ground truth and are computed over many queries, while live score is per-frame and immediate.

6.4 Step 3: Where database flow and UI flow meet

- **Database flow**: persisted map file provides descriptors + metadata + image paths.
- **Live flow**: each frame gets one descriptor, then a search backend retrieves the strongest candidates from stored descriptors.
- **UI flow**: renders decision state (MATCH/UNKNOWN), best match thumbnail, and top-k panel from stored `image_paths`.

So the runtime loop is:

load map once → process frame → retrieve candidates → threshold decision → visualize/export

6.5 Intuition summary

- The map is a compact searchable memory of known places.
- Online localization is repeated nearest-neighbor search in descriptor space.
- Threshold controls precision-recall tradeoff at runtime: higher threshold reduces false recognitions but increases unknowns.
- Top-k exists for operator confidence and debugging, even when only best match drives the binary decision.

7 Extension Points

If you want to change something, start here:

Goal	File to edit first
Add a new dataset	`datasets/load_dataset.py`
Add a new descriptor	`feature_extraction/` and `live_vpr/extractors.py`
Change benchmark metrics	`evaluation/metrics.py`
Change match selection	`matching/matching.py`
Change live map format	`live_vpr/database.py`
Change traversal recording	`live_vpr/capture.py`
Change live localization logic	`live_vpr/online.py`
Change search backend or ANN parameters	`live_vpr/search.py` and `configs/live_vpr.yaml`
Change YAML config defaults	`live_vpr/config.py` and `configs/live_vpr.yaml`
Change overlays / saved report images	`live_vpr/ui.py`
Change camera or stream handling	`live_vpr/sources.py`
Add a new live mode or CLI flag	`live_vpr_test.py`

8 Practical Mental Model

Almost everything in the repo reduces to the same conceptual loop:

image → descriptor → search → decision

The benchmark path adds evaluation metrics. The live path adds UI, capture sources, and saved reference maps. If you keep that mental model in mind, the rest of the codebase becomes much easier to navigate.

9 Known Diagram Rendering Pitfalls (and why these figures are stable now)

This guide uses three safeguards so the diagrams render predictably across editors and export flows:

- figures are pinned with `[H]` to avoid aggressive float reordering,
- architecture figures now use the higher-level `smartdiagram` package instead of hand-positioned TikZ nodes,
- diagram text widths are bounded so long file names do not spill outside the page box.

If a renderer still hides diagram graphics, compile with `pdflatex` or `xelatex` directly and ensure the standard diagram packages from TeX Live are available.